

PENETRATION TESTING REPORT

Documentation Version: 1.0

Target Name: TryHackMe — Grand Larceny Auto (Windows)

Version: v1.0

Prepared By: Security Assessment Team (Sameek Parajuli)

Date: July 26, 2026

Executive Summary

During the penetration test of the target application **Grand Larceny Auto**, a security assessment was performed across system binaries, assembly modules, and application database routines.

The evaluation identified a critical vulnerability involving **Unsafe Key Derivation and Hardcoded Cryptographic Logic** in the application assembly (GrandLarcenyAuto.dll), which allows an attacker to extract internal application secrets (SealedBlob) and bypass intended authorization checks (e.g., administrator/vault access).

Vulnerability Severity Breakdown

- **High Severity: 1**
- **Low Severity: 0**
- **Common Vulnerabilities Total: 1**
- **Total Number of Vulnerabilities: 1**

Severity Breakdown Graph:

[HIGH] (1)

[MEDIUM] (0)

[LOW] (0)

Vulnerability Summary Table

Vulnerability Title	CVSS v3.1 Score	Affected Endpoint / Module	Status
Hardcoded Cryptographic Secrets & Unsafe Vault Logic Leading to Full Secret Compromise	8.6 (High)	GrandLarcenyAuto.SafehouseVault (GrandLarcenyAuto.dll)	Open / Exploited

Detail of Technical Findings

Title: Hardcoded Cryptographic Secrets & Unsafe Vault Logic Leading to Account/Secret Compromise

Description:

SQL Injection and static parameter tampering in application modules often allow unauthenticated actors to retrieve sensitive vault secrets. In this application, the search/vault mechanism (GrandLarcenyAuto.SafehouseVault) uses a predictable key-derivation routine in CryptoUtil::DeriveKey.

The vault relies on a fixed salt format ("GLA::vault::key::v1::stars=" + stars) combined with a deterministic SHA-256 hash and XOR stream cipher to lock the vault secret (SealedBlob). Because the required star parameter is hardcoded (UnlockStars = 6), an attacker can reconstruct the SHA-256 key offline and decrypt the vault secret, leading to total compromise of administrative content and stored flag values.

Impact (CIA Triad Analysis):

- **Confidentiality: HIGH** — Unauthorized actors can fully extract and decrypt sensitive database/vault fields, compromising stored secrets and account tokens.
- **Integrity: HIGH** — Unauthenticated parameter manipulation bypasses core security controls without valid execution privileges.
- **Availability: LOW** — Direct system disruption is not guaranteed, but administrative takeover can lead to full access restriction.

Recommendations:

- **Implement Parameterized Input Validation:** Ensure all backend queries and parameters use strict input sanitation and parameterized calls.
- **Avoid Reversible Client-Side Encryption Secrets:** Never store static decryption keys or predictable key-derivation salts within compiled assemblies (.dll/.exe).
- **Implement Strong Access Control Policies:** Validate administrative and vault access checks on a secure server-side boundary rather than relying on local assembly states.

Affected Endpoint / Application:

- **Endpoint / Assembly:** GrandLarcenyAuto.SafehouseVault::SealedBlob
- **Target DLL:** data_GrandLarcenyAuto_windows_x86_64/GrandLarcenyAuto.dll
- **CVSS Score: 8.6 (CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:N)**

Steps of Reproduction:

1. Navigate to the application target directory containing GrandLarcenyAuto.dll.
2. Analyze the compiled bytecode using disassembly tools (ILSpy, dnSpy, or monodis) to locate SafehouseVault::cctor and CryptoUtil::DeriveKey.
3. Extract the hardcoded vault parameter (UnlockStars = 6) and salt string GLA::vault::key::v1::stars=6.
4. Run a reflection script (GetFlag.exe) or Python solver to extract SealedBlob and perform XOR decryption against the SHA-256 derived key.
5. Observe the decrypted sensitive vault payload returned in cleartext.

Proof of Concept (PoC):

Execution Output:

Plaintext

```
(venv) └─[sameekey@parrot]─[~/Desktop/tryhackme/GrandLarcenyAuto-windows]
└─ $ mcs /reference:GrandLarcenyAuto.dll GetFlag.cs
└─ $ mono GetFlag.exe
```

[+] Extracted SealedBlob (length: 33)

[***] FLAG: THM{h0tf1x3d_my_0wn_w4nt3d_l3v3l}

- **Fig 1:** *Successful extraction and decryption of internal vault flag via Mono reflection agent.*

References:

- **OWASP Top 10:** A02:2021 – Cryptographic Failures & A03:2021 – Injection
- **HackerOne Report Reference:** *Weak Key Derivation & Client-Side Secret Exposure*

Conclusion

The penetration test conducted on **Grand Larceny Auto** revealed a high-severity cryptographic design weakness. By leveraging static analysis and custom reflection tools, the security team successfully bypassed vault validation checks and retrieved confidential application data without requiring standard game execution. Implementing server-side authorization boundaries, secure key management, and parameterized inputs will prevent similar compromises in production environments.

Appendix: Attack Narrative

1. Initial Access & Reconnaissance:

The assessment began by auditing target binary dependencies in `data_GrandLarcenyAuto_windows_x86_64/`. The .NET assembly `GrandLarcenyAuto.dll` was identified as holding core business logic.

2. Reverse Engineering & Code Audit:

Decompilation revealed `GrandLarcenyAuto.SafehouseVault` initializing `SealedBlob` in `.cctor`. Inspection of `CryptoUtil` highlighted a predictable SHA-256 key derivation function accepting an integer `stars` parameter.

3. Exploitation & Secret Recovery:

By determining that `UnlockStars` was assigned a value of 6, a custom C# loader (`GetFlag.cs`) was compiled with Mono to invoke reflection calls against `GrandLarcenyAuto.dll`. The script calculated the key `$\text{SHA256}(\text{"GLA::vault::key::v1::stars=6"})$`, performed XOR decryption against `SealedBlob`, and extracted the root flag `THM{h0tf1x3d_my_0wn_w4nt3d_l3v3l}`.